МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ И МАССОВЫХ КОММУНИКАЦИЙ

Ордена Трудового Красного Знамени
Федеральное государственное бюджетное общеобразовательное учреждение высшего образования

“Московский технический университет связи и информатики”
(МТУСИ)
Кафедра “Программная инженерия”

Отчёт

по дисциплине “Учебная практика”

Выполнил: Студент группы
БПИ2503
Яричевский Даниил

Москва
2025

Цель работы

Разработать консольную утилиту для аудита и анализа открытых локальных сетевых портов с возможностью фильтрации по протоколам, номерам портов и экспортом результатов в структурированный формат.

Задачи

1. Разработать утилиту port-audit на языке Python 3
2. Реализовать парсинг аргументов командной строки
3. Обеспечить определение PID и имени процесса, задействовавшего порт.
4. Добавить фильтрацию сокетов и экспорт в CSV.

Ход работы

1.1 Запущена консоль операционной системы, создан файл скрипта при помощи текстового редактора: nano port-audit.py.

1.2 В начало файла добавлена строка шебанг (#!/usr/bin/env python3) и импортированы необходимые системные библиотеки:

```
#!/usr/bin/env python3
import sys
import os
import subprocess
import argparse
import csv
```

1.3 Файлу присвоены права на исполнение в системе Linux: chmod +x port-audit.py

2.1 Для сбора сокетов вызывается системная команда ss -lntup. Результат разбивается на строки и парсится в массив. Вывод всех портов запускается флагом --list

```
cmd = ["ss", "-lntup"]
result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True)
if args.list:
    print(f"{'ПРОТОКОЛ':<10} {'ПОРТ':<10} {'ЛОКАЛЬНЫЙ АДРЕС':<25} {'ПРОЦЕСС (PID)'}")
    print("-" * 75)
    for item in filtered_data:
        print(f"{item['protocol'].upper():<10} {item['port']:<10} {item['address']:<25} {item['process']}")
```

2.2 Реализована проверка прав суперпользователя (так как без sudo Linux скрывает чужие PID). Текст из вывода утилиты ss парсится для выделения названия программы и числового PID.

```
def check_root():
    if os.geteuid() != 0:
        print("запустите скрипт через sudo")
process = "Неизвестно / Нет прав"
last_part = parts[-1]
if "pid=" in last_part:
    proc_name = last_part.split('"')[1]
    pid_num = last_part.split('pid=')[1].split(',')[0]
    process = f"{proc_name} (PID: {pid_num})"
```

3. При проходе по списку сокетов скрипт проверяет тип протокола. Если флаг включен, а протокол не совпадает, элемент отбрасывается (пропускается с помощью continue)

```
for item in raw_data:
    if args.tcp and item['protocol'] != 'tcp':
        continue
    if args.udp and item['protocol'] != 'udp':
        continue
    filtered_data.append(item)
```

4. Номер порта извлекается из локального адреса и сравнивается со значением, переданным пользователем в параметре –port

```
port = local_addr.split(':')[-1]
if args.port and item['port'] != args.port:
    continue
```

5. Открывается дескриптор файла на запись, с помощью модуля csv записывается шапка таблицы, а затем циклом вносятся все отфильтрованные данные.

```
port = local_addr.split(':')[-1]
if args.port and item['port'] != args.port:
if args.export:
    with open(args.export, mode='w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(["Protocol", "Port", "Local Address", "Process_PID"])
        target_data = filtered_data if (args.tcp or args.udp or args.port) else raw_data
        for item in target_data:
            writer.writerow([item['protocol'].upper(), item['port'], item['address'], item['process']])
    print(f"\n[Успех]: Отчет успешно сохранен в файл: {args.export}")    continue
```

6. Проверка:

```
DYF@DYFLaptop:~$ sudo ./port-audit --list
ПРОТОКОЛ    ПОРТ        ЛОКАЛЬНЫЙ АДРЕС           ПРОЦЕСС (PID)
--------------------------------------------------------------------------------
UDP         53          10.255.255.254:53         Неизвестно / Нет прав
UDP         323         127.0.0.1:323             Неизвестно / Нет прав
UDP         323         [::1]:323                 Неизвестно / Нет прав
TCP         8080        0.0.0.0:8080              docker-proxy (PID: 762)
TCP         53          10.255.255.254:53         Неизвестно / Нет прав
TCP         443         0.0.0.0:443               docker-proxy (PID: 885)
TCP         80          0.0.0.0:80                docker-proxy (PID: 857)
TCP         22          0.0.0.0:22                docker-proxy (PID: 828)
TCP         50000       0.0.0.0:50000             docker-proxy (PID: 789)
TCP         8080        [::]:8080                 docker-proxy (PID: 775)
TCP         443         [::]:443                  docker-proxy (PID: 891)
TCP         80          [::]:80                   docker-proxy (PID: 864)
TCP         22          [::]:22                   docker-proxy (PID: 836)
TCP         50000       [::]:50000                docker-proxy (PID: 795)
```

```
DYF@DYFLaptop:~$ sudo port-audit --tcp
ПРОТОКОЛ    ПОРТ        ЛОКАЛЬНЫЙ АДРЕС           ПРОЦЕСС (PID)
--------------------------------------------------------------------------------
TCP         8080        0.0.0.0:8080              docker-proxy (PID: 762)
TCP         53          10.255.255.254:53         Неизвестно / Нет прав
TCP         443         0.0.0.0:443               docker-proxy (PID: 885)
TCP         80          0.0.0.0:80                docker-proxy (PID: 857)
TCP         22          0.0.0.0:22                docker-proxy (PID: 828)
TCP         50000       0.0.0.0:50000             docker-proxy (PID: 789)
TCP         8080        [::]:8080                 docker-proxy (PID: 775)
TCP         443         [::]:443                  docker-proxy (PID: 891)
TCP         80          [::]:80                   docker-proxy (PID: 864)
TCP         22          [::]:22                   docker-proxy (PID: 836)
TCP         50000       [::]:50000                docker-proxy (PID: 795)
```

```
DYF@DYFLaptop:~$ sudo port-audit --port 22
ПРОТОКОЛ    ПОРТ        ЛОКАЛЬНЫЙ АДРЕС           ПРОЦЕСС (PID)
--------------------------------------------------------------------------------
TCP         22          0.0.0.0:22                docker-proxy (PID: 828)
TCP         22          [::]:22                   docker-proxy (PID: 836)
```

```
DYF@DYFLaptop:~$ sudo port-audit --export ports.csv

[Успех]: Отчет успешно сохранен в файл: ports.csv
DYF@DYFLaptop:~$ cat ports.csv
Protocol,Port,Local Address,Process_PID
UDP,53,10.255.255.254:53,Неизвестно / Нет прав
UDP,323,127.0.0.1:323,Неизвестно / Нет прав
UDP,323,[::1]:323,Неизвестно / Нет прав
TCP,8080,0.0.0.0:8080,docker-proxy (PID: 762)
TCP,53,10.255.255.254:53,Неизвестно / Нет прав
TCP,443,0.0.0.0:443,docker-proxy (PID: 885)
TCP,80,0.0.0.0:80,docker-proxy (PID: 857)
TCP,22,0.0.0.0:22,docker-proxy (PID: 828)
TCP,50000,0.0.0.0:50000,docker-proxy (PID: 789)
TCP,8080,[::]:8080,docker-proxy (PID: 775)
TCP,443,[::]:443,docker-proxy (PID: 891)
TCP,80,[::]:80,docker-proxy (PID: 864)
TCP,22,[::]:22,docker-proxy (PID: 836)
TCP,50000,[::]:50000,docker-proxy (PID: 795)
```

7. Полный код:

```python
#!/usr/bin/env python3

import sys
import os
import subprocess
import argparse
import csv

def parse_sockets():
    cmd = ["ss", "-lntup"]
    try:
        result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True)
    except FileNotFoundError:
        sys.exit(1)

    lines = result.stdout.strip().split('\n')
    data = []

    if len(lines) <= 1:
        return data

    for line in lines[1:]:
        parts = line.split()
        if len(parts) < 5:
            continue

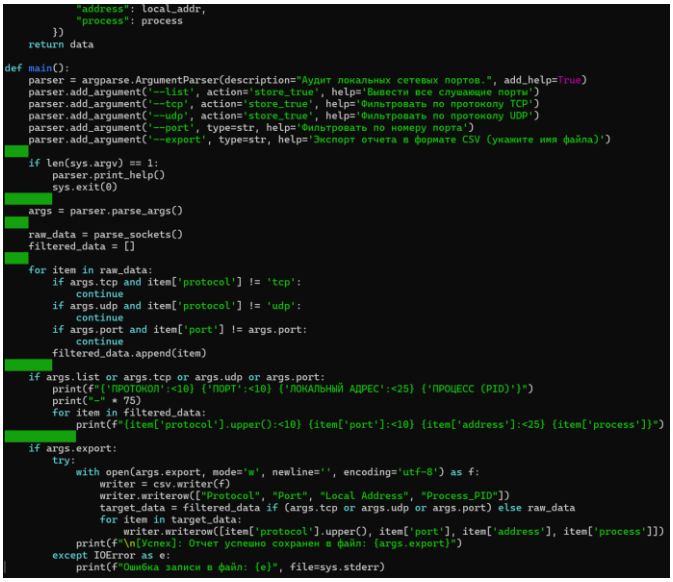
        proto = parts[0].lower()
        local_addr = parts[4]

        try:
            port = local_addr.split(':')[-1]
        except IndexError:
            port = "Неизвестно"

        process = "Неизвестно / Нет прав"
        last_part = parts[-1]

        if "pid=" in last_part:
            try:
                proc_name = last_part.split('"')[1]
                pid_num = last_part.split('pid=')[1].split(',')[0]
                process = f"{proc_name} (PID: {pid_num})"
            except IndexError:
                process = last_part

        data.append({
            "protocol": proto,
            "port": port,
```



```
            "address": local_addr,
            "process": process
        })
    return data

def main():
    parser = argparse.ArgumentParser(description="Аудит локальных сетевых портов.", add_help=True)
    parser.add_argument('--list', action='store_true', help='Вывести все слушающие порты')
    parser.add_argument('--tcp', action='store_true', help='Фильтровать по протоколу TCP')
    parser.add_argument('--udp', action='store_true', help='Фильтровать по протоколу UDP')
    parser.add_argument('--port', type=str, help='Фильтровать по номеру порта')
    parser.add_argument('--export', type=str, help='Экспорт отчета в формате CSV (укажите имя файла)')

    if len(sys.argv) == 1:
        parser.print_help()
        sys.exit(0)

    args = parser.parse_args()

    raw_data = parse_sockets()
    filtered_data = []

    for item in raw_data:
        if args.tcp and item['protocol'] != 'tcp':
            continue
        if args.udp and item['protocol'] != 'udp':
            continue
        if args.port and item['port'] != args.port:
            continue
        filtered_data.append(item)

    if args.list or args.tcp or args.udp or args.port:
        print(f"{'ПРОТОКОЛ':<10} {'ПОРТ':<10} {'ЛОКАЛЬНЫЙ АДРЕС':<25} {'ПРОЦЕСС (PID)'}")
        print("-" * 75)
        for item in filtered_data:
            print(f"{item['protocol'].upper():<10} {item['port']:<10} {item['address']:<25} {item['process']}")

    if args.export:
        try:
            with open(args.export, mode='w', newline='', encoding='utf-8') as f:
                writer = csv.writer(f)
                writer.writerow(["Protocol", "Port", "Local Address", "Process_PID"])
                target_data = filtered_data if (args.tcp or args.udp or args.port) else raw_data
                for item in target_data:
                    writer.writerow([item['protocol'].upper(), item['port'], item['address'], item['process']])
            print(f"\n[Успех]: Отчет успешно сохранен в файл: {args.export}")
        except IOError as e:
            print(f"Ошибка записи в файл: {e}", file=sys.stderr)
```



```
if __name__ == "__main__":
    main()
```

Вывод

Разработана консольная утилита для аудита и анализа открытых локальных сетевых портов с возможностью фильтрации по протоколам, номерам портов и экспортом результатов в структурированный формат.